SUPPORTING CHILDREN'S WRITING IN DANISH WITH TRANSFORMER MODELS

Mikkel Jordahn, Jonas Vestergaard Jensen, Søren Vejlgaard Holm, Michael Riis Andersen

Technical University of Denmark

ABSTRACT

In natural language processing applications, Large Language Models (LLMs) have become wildly popular due to their performance in many diverse settings. In this work, we investigate the potential of using Transformers for automated detailed feedback on children's writing in low-resource languages, in particular, Danish. The first step in such systems is to "decode" the children's writing to conventional writing, which can then be analyzed and used as a basis for providing fine-grained feedback on writing proficiency. We first show that it is possible to train Transformer architectures from scratch in low-data regimes to perform such decoding tasks and show that byte-pair encodings significantly outperform character-based encodings in this setting. We also compare the two encoding schemes in terms of noise sensitivity and out-of-distribution detection. Finally, we apply the proposed model to the downstream task of automated proficiency scoring.

***Index Terms*—** Natural Language Processing, Machine-Translation, Low-Resource Languages, Ed-Tech

1. INTRODUCTION

It is no secret that Large Language Models (LLMs) have seen an immense increase in interest since the release of ChatGPT, in large part due to their promise of being used in downstream natural language applications [1]. One such application is in e-learning platforms where the models can be fine-tuned for educational purposes such as supporting children's writing. Learning to write is not only essential for learning how to communicate in day-to-day life, but moreover lays the foundation for further education in a child's life. However, providing detailed feedback to support children's early writing is a time-consuming and resource-intensive process, especially in a class-room setting [2]. Furthermore, early writing is often characterized by incomplete text, unintentional text, phonetic spelling as well as lack of proper punctuation and spacing, which makes it highly non-trivial to analyze [3]. As such, the first step in automating feedback to support children's early writing is to translate incomplete and "noisy" student writing into "conventional" writing for subsequent analysis.

In this work, we cast the problem of predicting the intended writing of young students as a machine translation problem, where we "translate" from children's early writing (student texts) to conventional writing (teacher texts) in a sequence-to-sequence setting. In particular we train BART models [4] , a Transformer-based architecture, for the task. It has already been shown that in English, a resource-heavy language, it is possible to use machine learning to automatically support children's early-stage writing by fine-tuning pretrained LLMs [5]. In contrast, we investigate training LLMs architectures for Danish, a low-resource language, which is difficult for two reasons primarily. Firstly, there is significantly less data available, and secondly, there are few or no pretrained LLMs that perform as well as English LLMs [6]. Nevertheless, supporting and improving education is important for all languages, and hence, developing better language models for low-resource languages is an important problem from both technical and societal perspectives.

To investigate the extent to which it is possible to automatically support children's early-stage writing using machine learning in low-resource languages, we address the following three research questions. **RQ1:** To what degree can teacher texts be predicted from student texts in Danish? **RQ2:** Which type of text encoding is superior for this type of data? **RQ3:** Do these models have potential to be used in pipelines for e.g. automatic proficiency scoring as proposed in [7]?

1.1. Related Work & Contributions

Employing Transformer models for sequence-to-sequence tasks has been widely applied in the NLP literature, but with scarcity in teaching tasks, in particular in low-resource languages. Using machine learning for automated essay scoring (AES) is one of the few teaching tasks that has been investigated in [8], but in contrast to our work, the texts there are written by older students and in English. We rather look at the task of translating children's text in Danish as done in English in [5], but with the key difference that we do not use pre-trained models. Additionally, we also study using these models for the downstream teaching task of proficiency scoring as described in [3] using the methods described in [7] for automated proficiency scoring.

In this work we train FAIRSEQ's [9] BART models from scratch for correcting children's early-stage writing in a low-resource language, Danish, investigate their behaviour under

different noise levels and their ability to be utilized in downstream teaching tasks. The main contributions are:

1. We show that it is possible to train BART models without pretraining, investigate how the performance is influenced by model scaling, and show that byte-pair encodings (BPE) [10] encodings give the best performance.

2. We show that under increasing noise-levels, the likelihood of the decoded text decreases, indicating that the models are well-calibrated and "know what they don't know" even when they are not pretrained on large datasets.

3. We show that the translations provided by the models on children's texts can improve the accuracy of downstream teaching tasks, e.g. automated proficiency scoring of early-stage writing.

2. BACKGROUND AND METHODS

2.1. Sequence-to-Sequence Task

Throughout this work, we frame the task of predicting the teacher text from the student text as a "translation" task where the goal is to translate a student text of length m, $\mathbf{x} = [x_1, x_2, x_3, ..., x_m]$ to a teacher text $\mathbf{y} = [y_1, y_2, y_3, ..., y_h]$ of length h. Here, the elements of $\mathbf{x}$ and $\mathbf{y}$ are ordered tokens as provided by a tokenizer, T, using a vocabulary, V. As such, the translation task at hand is to predict the most probable translation $\mathbf{y}^*$ given some new student text $\mathbf{x}^*$, i.e. $\mathbf{y}^* = \arg\max_{\mathbf{y}} p(\mathbf{y}|\mathbf{x}^*)$. Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ and model M with parameters θ, we estimate θ by maximizing $p_\theta(\mathbf{y}|\mathbf{x})$ wrt. θ. We use the BART architecture implemented in FAIRSEQ [9], which consists of a bidirectional encoder and an auto-regressive decoder for which the likelihood of a translation $\mathbf{y}$ given $\mathbf{x}$ factorizes according to:

$$p(\mathbf{y}|\mathbf{x}) = \prod_{i=1}^{h} p(y_i|\mathbf{x}, y_{i-1}, ..., y_1), \tag{1}$$

i.e. the distribution of the next output sequence token y_i is conditioned on the input sequence $\mathbf{x}$ and previously output sequence tokens $[y_{i-1}, ..., y_1]$. Since each token belongs to a discrete vocabulary, the likelihood for a new token y_i in the sequence is naturally chosen to be a categorical likelihood, leading to the cross-entropy loss function. This means that for the n'th sample we have

$$\mathcal{L}(\mathbf{y}_n, \mathbf{x}_n) = -\sum_{i=1}^{h_n} \sum_{j=1}^{|V|} \mathbb{1}(y_i = j) \log p(y_i|\mathbf{x}, y_{i-1}, ..., y_1). \tag{2}$$

At prediction time, we use greedy search as a heuristic to maximize $p(\mathbf{y}|\mathbf{x})$ for translating a new sentence $\mathbf{x}^*$ token by token:

$$y_i^* = \arg\max_v p(y_i = v|\mathbf{x}^*, y_{i-1}^*, ..., y_1^*), \tag{3}$$

where v is the label for token $v \in V$.

2.2. Tokenizers

The first step is to tokenize an input sentence S according to some vocabulary V in order to produce the input sequence $\mathbf{x}$. The choice of the tokenizer can have an immense effect on the performance of the accompanying model. Tokenization strategies can vary from character-level to subword-level to word-level. The most trivial strategies are either to perform word embeddings based on a fixed word vocabulary V_{words} which is selected using a training text corpus or to simply use the individual characters, i.e. unigrams, of a chosen alphabet as the vocabulary V_{char}. Using a word-level vocabulary V_{words} has two benefits. Firstly, a word-based tokenizer captures word-level semantics that character-based models do not and secondly, the sequences of word-based tokenizers are significantly shorter which in turn increases the throughput of the accompanying model. However, using a character-level vocabulary V_{char} has the benefit of being able to handle all text without requiring unknown tokens, which V_{words} cannot. Byte-pair encodings (BPE) [10] combines the benefits of word-based and character-based tokenizations, and is one of the most commonly used tokenizers for LLMs. BPE tokenizers typically have a base vocabulary equivalent to V_{char}, but have additional vocabulary tokens that are chosen based on the most frequently observed subwords (e.g. bigrams, trigrams etc.) in a training corpus. The main downside with BPE tokenizers is that they have hyperparameters, e.g. granularity or size of vocabulary, that may not be trivial to choose.

2.3. Translation Evaluations

We evaluate the quality of the translations using the string metric called edit distance (ED) (also known as Levenshtein distance) on character-level as proposed in [11]. However, in order to avoid longer strings dominating the metrics, we normalize the ED with respect to the length of the string:

$$\text{NED}(\mathbf{s}^*, \mathbf{s}) = \frac{\text{ED}(\mathbf{s}^*, \mathbf{s})}{\max(|\mathbf{s}^*|, |\mathbf{s}|)}, \tag{4}$$

where $\mathbf{s}^*$ is the predicted output string and $\mathbf{s}$ is the true output string. In addition to NED, we track the calibration of the trained models, as model calibration can have large implications for downstream applications. We therefore compute the expected calibration error (ECE) and maximum calibration error (MCE) [12]. For ECE, the predictions for a test set are separated into B equally spaced bins according to the top-1

Model Name	# of Params	# of layers	Embed. dim	Attention heads
Base	140M	6	768	12
Light	29.99M	5	512	8
Small	6.34M	4	256	4
Tiny	0.31M	2	64	2

Table 1. Model sizes. Note here that # of layers is the same for encoder and decoder.

predicted confidence for each test point. The accuracy within each bin is then calculated, and the differences between these are then computed and summed. More precisely, the ECE is computed as:

$$\text{ECE}(\mathcal{D}_{test}) = \sum_{b=1}^{B} \frac{|\text{Bin}_b|}{N_{\text{test}}} |\text{acc}_b - \text{conf}_b|, \tag{5}$$

where $|\text{Bin}_b|$ is the number of samples in bin b, and acc_b and conf_b are the accuracy and confidence, respectively, within bin b, respectively. The MCE is simply the bin with the highest difference between the observed accuracy and confidence, i.e.:

$$\text{MCE}(\mathcal{D}_{test}) = \max_b |\text{acc}_b - \text{conf}_b|, \tag{6}$$

representing the worst-case calibration error.

3. DATA

In this work, we use the dataset compiled as a part of the Automated tracking of early-stage literacy (ATEL) research project [3]. The texts in the dataset were collected with parent consente for research purposes using a digital learning platform[1]. The dataset consists of pairs of sentences of student and teacher texts, where the student texts are written by children aged 6-10 and the teacher texts are "decoded" versions of the student text produced by class teachers. The training set, validation set and test set consist of $N_{\text{train}} = 380,191$, $N_{\text{valid}} = 8,872$ and $N_{\text{test}} = 10,000$ pairs, respectively. All data have been carefully anonymized prior to the analysis.

Due to the fact that the data has been collected from live teaching platform, the data contains a significant amount of "noise". Some of this is surmountable noise such as symbols, white-space characters, or emojis being present in the sentences. Other noise types give more fundamental issues which deteriorate the quality of the training data such as the children typing in both the teacher and student text fields or teachers using the platform in unintended ways, which is not easily resolvable at scale. For a more accurate evaluation, the test set has been carefully cleaned and curated by manually removing "noisy" instances, where e.g. the platform were used in unintended ways. Example of unintended use included case where the teachers used the "teaching field" to provide feedback to the student rather than writing the "decoded text". However, manual cleaning of the training and validation were infeasible due the size of the dataset.

[1] https://www.writereader.com/

4. EXPERIMENTS

To answer the research questions, we conduct four numerical experiments. In the first experiment, we investigate how model size and choice of tokenizer influence the predictive performance. Next, we conduct a noise sensitivity study, where we synthetically corrupt the ground-truth texts with noise, and observe how the best performing models behave as a function of noise level. In the third experiment, we compare the two models in terms of out-of-distribution detection. Finally, we study the models in the context of the pipeline described in [7] to investigate if Transformer architectures that have not been pre-trained on large datasets can still be used for downstream tasks.

4.1. Model Architectures and Tokenizers

We consider four different model sizes, see Table 1 for an overview. As we scale, we scale all parameters of the Transformer models, i.e. the number of layers in the encoder and decoder, the number of attention heads and the encoder embedding dimension are all simultaneously changed. We use the FAIRSEQ library [9] to train the models.

We experiment with two different types of tokenizers, namely character-based tokenization and BPE tokenization. The students texts are typically characterized by phonetic and transitional spelling, and hence, choosing a word-based tokenizer is impractical and not fitting for the task. The character-based tokenizer contains the Danish alphabet, whilst the BPE vocabulary is much larger. We train the BPE vocabulary on the training set with the default hyperparameters of the BPE tokenizer. We select the vocabulary size of the BPE tokenizer to be 1000, which also includes the letters in the Danish alphabet.

We use ADAM optimizer with a learning rate of 1e-4 and a Dropout rate of 0.1. Due to significant signs of overfitting (as evidenced by a large difference in train and validation loss) during training for the BPE Base model, we train another BPE Base model with much stronger regularization, namely with weight decay of 0.01 and a Dropout rate of 0.4.

	Mean NED	ECE	MCE	Mean time (s)	P[NED = 0]	P[NED = 0|Student NED=0]
Identity	0.23 ($\pm$0.00)	-	-	0.00 ($\pm$0.00)	0.17 ($\pm$0.01)	**1.00** (**$\pm$0.00**)
Identity + Post, Spell	0.23 ($\pm$0.00)	-	-	0.10 ($\pm$0.00)	0.19 ($\pm$0.01)	0.97 ($\pm$0.01)
Char Base	0.20 ($\pm$0.02)	0.01	0.06	0.45 ($\pm$0.01)	0.39 ($\pm$0.01)	0.86 ($\pm$0.02)
Char Light	0.21 ($\pm$0.02)	0.01	0.06	0.40 ($\pm$0.01)	0.38 ($\pm$0.01)	0.88 ($\pm$0.01)
Char Small	0.25 ($\pm$0.02)	0.01	0.04	0.33 ($\pm$0.01)	0.30 ($\pm$0.01)	0.87 ($\pm$0.02)
Char Tiny	0.97 ($\pm$0.10)	0.03	0.09	0.26 ($\pm$0.00)	0.14 ($\pm$0.01)	0.72 ($\pm$0.02)
BPE Base	0.19 ($\pm$0.02)	0.10	0.14	0.19 ($\pm$0.00)	0.43 ($\pm$0.01)	0.86 ($\pm$0.01)
BPE Base Reg.	**0.16** (**$\pm$0.01**)	0.12	0.20	0.18 ($\pm$0.00)	**0.45** (**$\pm$0.01**)	0.87 ($\pm$0.02)
BPE Light	0.18 ($\pm$0.01)	0.10	0.13	0.16 ($\pm$0.00)	0.42 ($\pm$0.01)	0.88 ($\pm$0.02)
BPE Small	0.20 ($\pm$0.02)	0.09	0.10	0.14 ($\pm$0.00)	0.38 ($\pm$0.01)	0.88 ($\pm$0.02)
BPE Tiny	0.27 ($\pm$0.03)	0.10	0.15	0.10 ($\pm$0.00)	0.21 ($\pm$0.01)	0.88 ($\pm$0.02)

Table 2. Performance metrics on test set. Reported values are means across the test set, and in parenthesis standard errors of the mean are reported. Char and BPE indicates character-based and BPE tokenizations respectively. Reg. denotes the model trained with stronger regularization.

In all cases, we employ early stopping based on the validation loss. As a references, we compare all models to an "Identity" model, which is simply an identity map, as well as an identity map followed by simple postprocessing with a Hunspell[2] spellchecker.

4.2. Text Reconstruction Task

Table 2 shows the evaluation metrics after training the models. In terms of NED, we note the general pattern that for both character-based and BPE-based models the performance improves as we increase model size up to that of the Base models, where regularization becomes important to combat overfitting due to the increased model flexibility. Finally, we also note how all the models outperform the baseline except for both Tiny configurations and the character-based Small configuration which likely is due to insufficient model complexity.

With regards to ECE and MCE, we firstly note the difference between the character-based and BPE tokenized models. This significant difference in these metrics is due to the fact that the space space for the BPE model is significantly larger per token compared to the character-based models, and therefore these cannot be directly compared to each other. Nevertheless interestingly we note that, once again ignoring the Tiny configuration, that when we increase model size, model calibration decreases, which is likely related to the well-known phenomenon of over-confidence in over-parameterized models.

We also compare the models in terms of metrics that are important for practical use of the models. We compute the fraction of translations that have a NED of 0, $P(\text{NED} = 0)$, the fraction of translations that have a NED of 0 given the student text was already correct, $P(\text{NED} = 0|\text{Student NED} = 0)$. Finally, we also consider the prediction time, i.e. the average time for decoding a student text . We note that the $P(\text{NED} = 0)$ significantly increases in comparison to the baselines whilst no model significantly deteriorates $P(\text{NED} = 0|\text{Student NED} = 0)$ except the Tiny configurations. Finally, we note how the mean prediction times of the BPE models are almost half that of the character-based models.

In the remaining experiments, we will study the performing character-based model (Char Base) and BPE-based model (BPE Base reg.) in greater details.

4.3. Noise sensitivity and robustness

Next, we investigate model performance under increasing amounts of noise to determine their robustness. We do this by creating synthetic datasets in which we take the teacher sentence $\mathbf{y}$, and inject varying amount of noise into it to create a synthetic $\hat{\mathbf{x}}$. We inject noise by randomly replacing characters in the teacher text with probability α. We sweep the noise rate across 21 values from 0 to 0.20 creating 21 synthetic datasets and evaluate the NED for each of the datasets. In addition to the NED, we also track the mean likelihood normalized wrt. output length, the respective models assign to translations they perform on a given dataset, i.e. $p(\mathbf{y}^*|\mathbf{x}^*)$, to study to which degree the models "know what they don't know". We show the result of this experiment in Figure 1. Here we note that for no-to-low levels of noise, both models have a consistently very low NED and a very high assigned likelihood. As the noise levels are increased, the NED increases with the mean likelihood decreasing, although with the BPE Base

[2]https://github.com/pyhunspell/pyhunspell

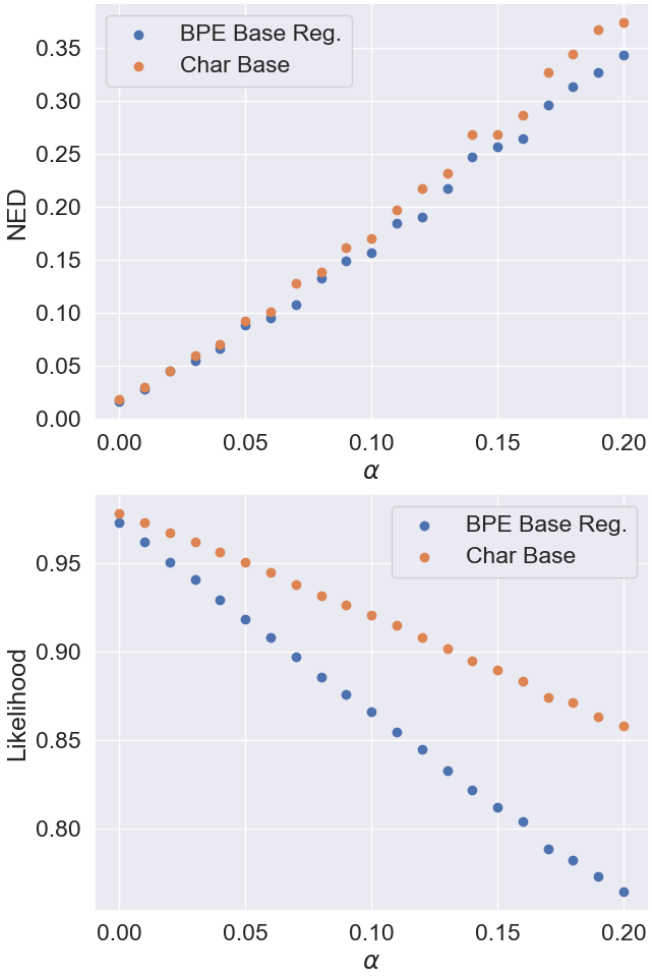

Fig. 1. Robustness to noise: The top panel shows the NED metric as a function of noise probability α, whereas the bottom panel shows the mean likelihood of the decoding as a function of α.

model performing the best as the noise level are increased. Moreover, the decrease in likelihood of the BPE Base Reg. model is also more in line with the increase in NED as the noise rate increases.

4.4. Out-of-distribution detection

In practical applications, it is of great value to be able to flag "poor translations", which can occur for student texts "far away" from the training data. Motivated by the desire to reject "poor translations" automatically, we focus more explicitly on out-of-distribution (OOD) detection in this experiment. For this purpose, we consider five OOD datasets, where the models are expected to yield low-quality predictions, and then we investigate to what degree the models can detect the text instances from these OOD datasets via the normalized likelihood introduced above.

#	OOD Dataset	BPE	Char
1	Test set w. shuffled characters	0.98	0.98
2	Test set w. shuffled words	0.77	0.71
3	Test set w. reversed texts	0.98	0.98
4	Random letters from Danish alphabet	0.99	1.00
5	English texts [5]	0.96	0.96

Table 3. AUROC results for out-of-distribution detection across five different OOD datasets.

The five OOD datasets are described in the following: 1) the test set (as described in Section 3), but where the ordering of the characters has been shuffled, 2) the same test set, but where the ordering of the words has been shuffled, 3) the same test set, but where each text has been reversed, 4) the same test set, where each letter is replaced with a random letter from the danish alphabet, 5) a corpus of students texts in English [5]. Note that the first three OOD datasets preserve the character-level unigram distributions for each text.

For each OOD dataset, we compute the model translations as well as the normalized likelihoods for a subset of 2500 texts and we compute the model translation for 2500 texts in the original and unperturbed test set. Phrasing the OOD detection problem as binary classification, we can use the AUROC metric [13] to quantify the OOD-detection performance. The results are shown in Table 3.

With AUROC scores of ≥ 0.98, it is seen that is generally very easy for the models to identify the texts consisting of random letters (#4) as well as texts that have been either reversed (#3) or shuffled on character level (#1. Similarly, the AUROC scores for the English texts (#5) are 0.96 for both models. Generally, the OOD-detection performance for both models is virtually the same for all OOD-datasets except the dataset with danish texts with shuffled words (#), where the BPE-based model is significantly better at identifying the OOD texts. The AUROC score for this specific OOD dataset is 0.77 and 0.71 for the BPE-based model and the Character-based models, respectively, and is seen to be significantly lower compared to other datasets.

4.5. Automatic Proficiency Scoring

The purpose of this section is to investigate the feasibility of using the proposed models for the downstream task of automatic proficiency scoring. More specifically, Andersen et al. (2023) proposed a framework for automatic proficiency scoring of early-stage writing, where texts are quantified using three dimensions: 1) sentence construction, 2) text construction, and 3) use of modifiers [7]. However, the framework requires access to teacher texts for language analysis, and this, in turn, limits the applicability at scale. Therefore, we investigate to what degree the predicted teacher texts can be used as surrogates for actual teacher texts, when estimat-

ing the three-dimensional proficiency scores. We consider the dataset consisting of $N = 803$ pairs of student texts $\mathbf{x}_n$ and teacher texts $\mathbf{y}_n$ described by Kabel et al. (2022) [3]. We conduct the experiment as follows. First, we use the framework to evaluate proficiency scores for each of the 803 texts using the actual teacher texts $\mathbf{y}_n$, yielding proficiency estimates $\{\theta_n^i\}_{n=1}^N$, where $i \in \{1, 2, 3\}$ indexes the dimension. We consider these the gold standard proficiency scores. Next, we feed the student texts from the dataset to the Transformer models to produce estimates of the teacher texts $\hat{\mathbf{y}}_n$, and then we use the teacher texts $\hat{\mathbf{y}}$ to evaluate the proficiency scores to obtain $\hat{\theta}_n^i$. For reference, we also compute the proficiency scores using the raw student texts $\mathbf{x}_n$ yielding $\{\phi_n^i\}_{n=1}^N$. Finally, we compute the mean-squared error (MSE) between the scores obtained from student texts and teacher texts, i.e. $|\theta_n^i - \phi_n^i|^2$, and the MSE between the scores obtained from the translated texts and teacher texts, i.e. $|\theta_n^i - \hat{\theta}_n^i|^2$. The results can be seen in Table 4. It is seen that the texts translated with Char Base and BPE Base Reg. leads to substantial improvements on the MSE in comparison to the scores computed on the raw student texts. Interestingly, whilst both models improve on all characteristics of the texts, BPE Base Reg. performs the best and in particular on the characteristics of Sentence and Text Construction.

5. DISCUSSION AND CONCLUSIONS

5.1. Predicting the Teacher Texts

We posed the research question (**RQ1**) *To what degree can teacher texts be predicted from student texts in Danish without pretrained models?*, and investigated this by training BART architectures of varying size using cross-entropy loss. In the results in Table 2, we notice how most model configurations beat the baselines, and how the NED metrice generally decreased as we increased model scale. However, our results also indicate that upscaling the models increase over-confidence especially in the BPE based models as indicated by the MCE metrics. Nevertheless, we conclude that it is possible to predict teacher texts from students without using pretrained models. It could be of interest to investigate if either pretraining on denoising tasks first in Danish or using other methods for getting better model calibration, such as Bayesian Neural Networks, could improve such apparent calibration issues. Moreover, based on the results in [5], there is an indication that pretraining with denoising tasks fit this problem type well. We leave both of these investigations for future works.

5.2. BPE Outperforms Character-based Tokenizers

In addition to the effect of scaling model size, we investigated the effect of different tokenizers **(RQ2)**. We experimented with BPE and character-based tokenizers, excluding word-based encodings due to their apparent problem with highly noisy text data. Across all evaluations it became apparent that the BPE tokenizer was superior. Firstly, in Table 2, controlling for model size, the BPE models outperformed character-based models on all metrics. Secondly, in the noise sensitivity experiment, the NED also appeared to deteriorate at a faster rate for the character-based model, than the BPE based model, indicating that even in low-data regimes BPE based models may be more robust to noise. In addition, the mean prediction time of the BPE models were less than half of that of the character-based models, an unsurprising result considering the scaling of Multi-Head attention layers with input sequence length. Based on the ECE and MCE metrics, it could appear that the character-based models in fact were better calibrated, but the results from the noise-simulation experiment indicates otherwise as the BPE based models show a faster decline in likelihood of translations which aligns better with the increase in NED, i.e. the BPE models empirical performance is better reflected by the decrease in likelihood.

5.3. Danish LLMs show promise for use in Educational Technology

Finally we investigated whether the models trained for translation, could be used in pipelines for automated proficiency scoring **(RQ3)**. We used the best performing models, i.e. BPE Base Reg. and Char Base in these pipelines, and showed in Table 4 how the MSE of the translated texts provided characteristic metrics closer to those of the teacher, with the BPE based model once again performing the best.

The $P(\text{NED} = 0)$ and $P(\text{NED} = 0|\text{Student NED} = 0)$ metrics also provided important evidence for practical applications. Firstly, the increase in $P(\text{NED} = 0)$ from 0.17 for the baseline to 0.45 for the best model, i.e. BPE Base Reg, is a significant increase in the fraction of texts in which the teacher would not have to perform any translations. The result that $P(\text{NED} = 0|\text{Student NED} = 0)$ does not decrease is also significant, as it means that using the Transformer models would not introduce *more* work for teachers. Based on these results we conclude that there is indeed potential for using Transformer models as teacher support for automating feedback on children's early writing.

6. REFERENCES

[1] Yiheng Liu, Tianle Han, Siyuan Ma, Jiayue Zhang, Yuanyuan Yang, Jiaming Tian, Hao He, Antong Li, Mengshen He, Zhengliang Liu, Zihao Wu, Lin Zhao, Dajiang Zhu, Xiang Li, Ning Qiang, Dingang Shen, Tianming Liu, and Bao Ge, "Summary of chatgpt-related research and perspective towards the future of large language models," *Meta-Radiology*, vol. 1, no. 2, pp. 100017, 2023.

Model	Sentence Construction	Text Construction	Modifier Use
Identity	0.710	0.372	0.311
Char Base	0.513	0.290	0.160
BPE Base Reg.	0.375	0.160	0.120

Table 4. MSE of writing characteristics of students, Char Base and BPE Base Reg. translations vs. teacher texts.

[2] Drita Saliu-Abdulahi, Glenn Ole Hellekjær, and Frøydis Hertzberg, "Teachers (formative) feedback practices in efl writing classes in norway," 2017.

[3] Kristine Kabel, Jesper Bremholm, and Jeppe Bundsgaard, "A framework for identifying early writing development," *Writing and Pedagogy*, vol. 13, no. 1-3, pp. 51–87, Jul. 2022.

[4] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 7871–7880.

[5] Jonas Vestergaard Jensen, Mikkel Jordahn, and Michael Riis Andersen, "Neural machine translation for automated feedback on children's early-stage writing," in *Proceedings of the 5th Northern Lights Deep Learning Conference (NLDL)*. 09–11 Jan 2024, vol. 233 of *Proceedings of Machine Learning Research*, pp. 104–112, PMLR.

[6] Andreas Kirkedal, Barbara Plank, Leon Derczynski, and Natalie Schluter, "The lacunae of Danish natural language processing," in *Proceedings of the 22nd Nordic Conference on Computational Linguistics*, Turku, Finland, Sept.–Oct. 2019, pp. 356–362, Linköping University Electronic Press.

[7] Michael Riis Andersen, Kristine Kabel, Jesper Bremholm, Jeppe Bundsgaard, and Lars Kai Hansen, "Automatic proficiency scoring for early-stage writing," *Computers and Education: Artificial Intelligence*, vol. 5, pp. 100168, 2023.

[8] Dadi Ramesh and Suresh Kumar Sanampudi, "An automated essay scoring systems: A systematic literature review," *Artif. Intell. Rev.*, vol. 55, no. 3, pp. 2495–2527, mar 2022.

[9] Myle Ott, Sergey Edunov, Alexei Baevski, Angela Fan, Sam Gross, Nathan Ng, David Grangier, and Michael Auli, "fairseq: A fast, extensible toolkit for sequence modeling," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, Minneapolis, Minnesota, June 2019, pp. 48–53, Association for Computational Linguistics.

[10] Rico Sennrich, Barry Haddow, and Alexandra Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Berlin, Germany, Aug. 2016, pp. 1715–1725, Association for Computational Linguistics.

[11] Vladimir I Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," in *Dokl. Akad. Nauk SSSR*, 1965, vol. 163, pp. 845–848.

[12] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht, "Obtaining well calibrated probabilities using bayesian binning," in *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*. 2015, AAAI'15, p. 2901–2907, AAAI Press.

[13] Jingkang Yang, Pengyun Wang, Dejian Zou, Zitang Zhou, Kunyuan Ding, WENXUAN PENG, Haoqi Wang, Guangyao Chen, Bo Li, Yiyou Sun, Xuefeng Du, Kaiyang Zhou, Wayne Zhang, Dan Hendrycks, Yixuan Li, and Ziwei Liu, "Openood: Benchmarking generalized out-of-distribution detection," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds. 2022, vol. 35, pp. 32598–32611, Curran Associates, Inc.